

گزارش سوال ۳ عملی - تمرین ۸ - سیستم عامل

این تمرین را بر روی فایل‌های تمرین ۷ که از BitMap استفاده می‌کنند ادامه می‌دهیم.

گام‌های تمرین:

۱. مشکلات/احتمال در مورد استفاده همزمان چند کاربر از این سامانه را نام ببرید.

برنامه‌ای که تا اینجا نوشته ایم، برای استفاده یک کاربر در آن واحد طراحی شده و به همین دلیل در صورتی که به طور همزمان توسط چندین کاربر بر روی یک فایل سیستم اجرا شود، ممکن است مشکلاتی به وجود بیاید. توجه داریم که در صورتی که فایل سیستمی که کاربران از آن‌ها استفاده می‌کنند متفاوت باشد، مشکلی به وجود نمی‌آید؛ چرا که هر یک در حال کار با یک فایل مجزا بر روی دیسک کامپیوتر هستند.

در واقع، برای ایجاد تغییراتی مانند ایجاد فایل، نوشتن درون فایل، حذف فایل، تغییر اندازه فایل، ایجاد و یا حذف کاربر و گروه و ... ، ابتدا وضعیت فعلی فایل سیستم خوانده و بررسی می‌شود (مرحله ۱)، سپس تغییرات مورد نظر بر آن اعمال شده (مرحله ۲) و مجدداً درون فایل سیستم نوشته می‌شود (مرحله ۳). به همین علت، ممکن است در حالی که یک پردازنده در مرحله ۲ قرار دارد، پردازنده دیگر مرحله ۱ را انجام دهد، سپس پردازنده اول مرحله ۳ را انجام دهد. در این روند، پردازنده دوم تغییراتی که پردازنده اول در مرحله ۳ اعمال کرده است را نادیده خواهد گرفت و تغییرات خود را بر روی آن overwrite خواهد کرد.

۲. تلاش کنید یکی از مثال‌هایی که در بخش قبل نام بردید را روی سامانه خود اجرا کنید و نتایج آن را گزارش کنید.

در این بخش به بررسی دقیق‌تر مشکلاتی که می‌تواند از اجرای همزمان دستور ایجاد فایل جدید توسط دو کاربر ایجاد شود می‌پردازیم. تابع ایجاد فایل به صورت زیر است:

```

uint32_t create_file(const char *name) {
>   if (strlen(name) >= FILENAME_MAX_LEN) { ...
>   }
>   if (find_file(name, NULL) != 0) { ...
>   }

    // Allocate 1 block for metadata
    uint32_t off = fs_alloc(); ← 1
    if (off == 0) return 0;

    file_entry_t ent = {0};
    strcpy(ent.name, name);
>   ent.perm = (permission_t){ ...
>   };
    ent.size = 0;
    save_file_entry(off, &ent);

    superblock_t sb;
    load_superblock(&sb); ← 2
    sb.file_count++;
    save_superblock(&sb);

    return off;
}

```

در بخش 1، از تابع `fs_alloc` آدرس اولین `block` خالی و قابل استفاده را می‌گیریم و حالت آن را به «استفاده شده» تغییر می‌دهیم. در صورتی که قبل از اینکه پرده اول حالت `block` مورد نظر را به «استفاده شده» تغییر داده باشد، پرده دوم تابع `fs_alloc` را فراخوانی کند، آدرس همان `block` به پرده دوم نیز داده شده و هر دو پرده قصد ایجاد دو فایل متفاوت را در یک `block` یکسان خواهند داشت.

همچنین در بخش 2، در صورتی که قبل از اجرا شدن `save_superblock` توسط پرده اول، `load_superblock` توسط پرده دوم فراخوانی شود، تعداد فایل‌ها به جای اینکه ۲ عدد افزایش بیابد، فقط ۱ عدد افزایش پیدا می‌کند.

ابتدا به طور دستی حالت بالا را بررسی کردیم، اما از آنجا که سرعت اجرای توابع بسیار بیشتر از سرعت ما در اجرای سریع متوالی هر دو دستور بود، مشکلی پیش نیامد و هر دو فایل به درستی ایجاد شدند:

```

o martin@Matin:~/Term5/OS/HW/8$ ./myfs
FS CLI - type 'help' for commands.
myfs> init filesystem.db
Loaded existing filesystem filesystem.db
myfs> open file1 1
File created and opened: file1
myfs> fs_stats
Unknown command: fs_stats
Type 'help' for command list.
myfs> fs_stats

----- Filesystem statistics: -----
Total size: 134217728 bytes (128.00 MB)
Block size: 4096 bytes (4.00 KB)
File count: 2
Used space: 8192 bytes (8.00 KB)
Free space: 134209536 bytes (127.99 MB)
-----

myfs>
o martin@Matin:~/Term5/OS/HW/8$ ./myfs
FS CLI - type 'help' for commands.
myfs> init filesystem.db
Loaded existing filesystem filesystem.db
myfs> open file2 1
File created and opened: file2
myfs> fs_stats

----- Filesystem statistics: -----
Total size: 134217728 bytes (128.00 MB)
Block size: 4096 bytes (4.00 KB)
File count: 2
Used space: 8192 bytes (8.00 KB)
Free space: 134209536 bytes (127.99 MB)
-----

myfs>

```

برای شبیه‌سازی حالتی که هر دو دستور به طور دقیقاً همزمان اجرا شوند، در تابع مورد نظر وقفه‌ای افزودیم:

```

superblock_t sb;
load_superblock(&sb);
sleep(10);          // wait for 10 seconds
sb.file_count++;
save_superblock(&sb);

```

با وجود اینکه پیام ایجاد موفق هر دو فایل را مشاهده می‌کنیم، تعداد فایل‌های موجود در فایل سیستم تنها یکی در نظر گرفته شده است و این یکی از مشکلاتی است که در اجرای همزمان می‌تواند رخ دهد:

```

o martin@Matin:~/Term5/OS/HW/8$ ./myfs
FS CLI - type 'help' for commands.
myfs> init fs
Created new filesystem 'fs' (128MB, Bitmap Allocator)
myfs> open file1 1
File created and opened: file1
myfs> fs_stats

----- Filesystem statistics: -----
Total size: 134217728 bytes (128.00 MB)
Block size: 4096 bytes (4.00 KB)
File count: 1
Used space: 4096 bytes (4.00 KB)
Free space: 134213632 bytes (128.00 MB)
-----

myfs>
o martin@Matin:~/Term5/OS/HW/8$ ./myfs
FS CLI - type 'help' for commands.
myfs> init fs
Loaded existing filesystem fs
myfs> open file2 1
File created and opened: file2
myfs> fs_stats

----- Filesystem statistics: -----
Total size: 134217728 bytes (128.00 MB)
Block size: 4096 bytes (4.00 KB)
File count: 1
Used space: 4096 bytes (4.00 KB)
Free space: 134213632 bytes (128.00 MB)
-----

myfs>

```

۳. روی بخش های مختلف سامانه خود قفل بگذارید به طوری که دو کاربر به طور همزمان بتوانند بدون ایجاد مشکل روی سامانه از آن استفاده کنند.

می دانیم مشکل زمانی رخ می دهد که چند پردازش به طور همزمان با یک فایل (فایل سیستم کلی ما) کار کنند. برای جلوگیری از بروز چنین مشکلاتی، هسته سیستم عامل راه حلی دارد به نام Advisory file locks (fcntl). این نوع lock ها در per-process scope هستند که نیاز ما است، چرا که هر کاربر در یک shell یک process جدید ایجاد می کند و مشکل در race-condition بین process ها است و نه بین thread ها. ابتدا توابع lock و unlock را به این شکل تعریف می کنیم:

```
// ----- HW8 -----
// ----- locking -----
static void fs_lock(void) {
    struct flock fl = {0};
    fl.l_type = F_WRLCK;           // What kind of lock
    fl.l_whence = SEEK_SET;        // write lock
    fl.l_start = 0;                // Offset in file
    fl.l_len = 0;                  // Length (0 = entire file)
    // F_SETLKW: Wait until lock becomes available
    fcntl(fileno(g_fs_file), F_SETLKW, &fl);
}

static void fs_unlock(void) {
    struct flock fl = {0};
    fl.l_type = F_UNLCK;
    fl.l_whence = SEEK_SET;
    fl.l_start = 0;
    fl.l_len = 0;
    fcntl(fileno(g_fs_file), F_SETLK, &fl);
}
```

سپس از آنها درون توابعی که کار با فایل و نوشتن درون فایل سیستم دارند (ایجاد و حذف فایل، نوشتن) استفاده می‌کنیم.

برای مثال تابع `create_file` به این شکل در می‌آید:

```
uint32_t create_file(const char *name) {
    fs_lock();    // 🔒 START

    if (strlen(name) >= FILENAME_MAX_LEN) { ...
    if (find_file(name, NULL) != 0) { ...
    uint32_t off = fs_alloc();
    if (off == 0) { ...
    file_entry_t ent = {0};
    strcpy(ent.name, name);
    ent.perm = (permission_t){ ...
    ent.size = 0;
    save_file_entry(off, &ent);

    superblock_t sb;
    load_superblock(&sb);
    sleep(10);    // wait for 10 seconds (HW8)
    sb.file_count++;
    save_superblock(&sb);

    fs_unlock();    // 🔓 END
    return off;

fail:
    fs_unlock();
    return 0;
}
```

اکنون مجدداً گام ۲ این تمرین را انجام داده و مشاهده می‌کنیم که هر دو فایل به درستی ایجاد می‌شوند:

```
o martin@Martin:~/Term5/OS/HW/8$ ./myfs
FS CLI - type 'help' for commands.
myfs> init fs3
Created new filesystem 'fs3' (128MB, Bitmap Allocator)
myfs> open file1 1
File created and opened: file1
myfs> fs_stats

----- Filesystem statistics: -----
Total size: 134217728 bytes (128.00 MB)
Block size: 4096 bytes (4.00 KB)
File count: 2
Used space: 8192 bytes (8.00 KB)
Free space: 134209536 bytes (127.99 MB)
-----

myfs> █
```

```
o martin@Martin:~/Term5/OS/HW/8$ ./myfs
FS CLI - type 'help' for commands.
myfs> init fs3
Loaded existing filesystem fs3
myfs> open file2 1
File created and opened: file2
myfs> fs_stats

----- Filesystem statistics: -----
Total size: 134217728 bytes (128.00 MB)
Block size: 4096 bytes (4.00 KB)
File count: 2
Used space: 8192 bytes (8.00 KB)
Free space: 134209536 bytes (127.99 MB)
-----

myfs> █
```

علاوه بر اینکه تعداد فایل‌ها و فضای مصرف شده صحیح است، به کمک مدت زمان اجرا نیز می‌توانیم به صحت عملکرد lock ها پی ببریم. در گام ۲ که از locking استفاده نکرده بودیم، هر دو پردازش به طور موازی تابع ایجاد فایل را اجرا می‌کردند و پس از ۱۰ ثانیه، هر دو پیام file create and opened را پرینت می‌کردند. اما در این گام، پردازش ۱ پس از ۱۰ ثانیه این پیام را چاپ کرد، و پردازش ۲ پس از ۱۰ ثانیه دیگر (یعنی در کل پس از گذشت ۲۰ ثانیه) پیام را چاپ کرد. و این یعنی تا زمانی که اجرای تابع در پردازش ۱ تمام نشده بود، پردازش ۲ نمی‌توانست تابع خود را اجرا کند، چرا که فایل fs3 توسط kernel, lock شده بود.

در این تمرین ما از lock کردن کل فایل فایل سیستم به کمک kernel استفاده کردیم و در ابتدای اجرای توابع خاص (ایجاد فایل و ...). فایل را lock و در انتهای تابع آن را unlock کردیم. در حالی که می توانستیم صرفاً در بخش هایی از تابع که امکان وقوع race-condition در آن ها هست (اشاره گرهای قرمز در تصویر کد گام ۲) از lock کردن فایل استفاده کنیم. به طور کلی یکی از اصلی ترین نقاط ضعف این نوع lock استفاده شده، این است که هرگاه یکی از پردازها در حال اجرای یکی از توابع حساس (توابع دارای lock) باشد، هیچ یک از پردازهای دیگر نمی توانند تابع حساسی را اجرا کنند، چرا که کل فایل سیستم lock شده است. برای مثال، اگر پردازهای در حال نوشتن بر روی فایل ۱ باشد، پرداز ۲ نمی تواند بر روی فایل ۲ بنویسد.

همچنین، این روش lock کردن کل فایل سیستم، برای استفاده همزمان تعداد زیادی کاربر از یک فایل سیستم مناسب نیست، چرا که با افزایش تعداد کاربران، زمان انتظار بالا رفته و ممکن است starvation رخ دهد. در اینجا ما امکان ایجاد تعداد زیادی کاربر و نشان دادن تجمیع زمان های انتظار اجرای توابع نداشتیم، به همین دلیل، برای نشان دادن آن در عمل و با ۲ کاربر، زمان انتظار ۱۰ ثانیه ای را برای اجرای تابع در نظر گرفتیم که شبیه سازی از زمانی است که منتظر اتمام اجرای تابع هزاران کاربر دیگر باشیم و دیدیم که کاربر دوم باید ۲۰ ثانیه منتظر باشد. بدیهی است که چنین زمان انتظاری برای کاربر قابل قبول نخواهد بود.

از دیگر نقاط ضعف این نوع lock می توان به نبود قفل های ریزدانه (Fine-grained locking) اشاره کرد. در این پیاده سازی، قفل به صورت یکپارچه روی کل فایل سیستم اعمال می شود و تفاوتی بین قفل فایل های مختلف یا حتی بلاک های مختلف در نظر گرفته نشده است. این در حالی است که در یک طراحی بهینه تر، می توان برای هر فایل (یا حتی هر inode یا block قفل جداگانه ای در نظر گرفت تا کاربران بتوانند همزمان روی فایل های متفاوت کار کنند.

همچنین، در صورتی که در یکی از توابع به هر دلیلی مسیر اجرای برنامه به fs_unlock نرسد (مثلاً بازگشت زودهنگام یا خطای برنامه نویسی)، کل فایل سیستم برای سایر کاربران قفل باقی می ماند تا زمانی که فرآیند خاتمه یابد. هرچند kernel در صورت پایان فرآیند قفل را آزاد می کند، اما در زمان اجرای برنامه این مسئله می تواند باعث توقف کامل سیستم شود.